

WebSocket

WebSocket is a computer communications protocol, providing a bidirectional communication channel over a single Transmission Control Protocol (TCP) connection. The protocol was standardized by the IETF as RFC 6455 (<https://www.rfc-editor.org/rfc/rfc6455>) in 2011. The current specification allowing web applications to use this protocol is known as *WebSockets*.^[1] It is a living standard maintained by the WHATWG and a successor to *The WebSocket API* from the W3C.^[2]

WebSocket is distinct from HTTP used to serve most webpages. Although they are different, RFC 6455 (<https://www.rfc-editor.org/rfc/rfc6455>) states that WebSocket "is designed to work over HTTP ports 443 and 80 as well as to support HTTP proxies and intermediaries", making the WebSocket protocol compatible with HTTP. To achieve compatibility, the WebSocket handshake uses the HTTP Upgrade header^[3] to change from the HTTP protocol to the WebSocket protocol.

The WebSocket protocol enables full-duplex interaction between a web browser (or other client application) and a web server with lower overhead than half-duplex alternatives such as HTTP polling, facilitating real-time data transfer from and to the server. This is achieved by providing a standardized way for the server to send content to the client without being first requested by the client, and allowing messages to be exchanged while keeping the connection open. In this way, a two-way ongoing conversation can take place between the client and the server. The communications are usually done over TCP port number 443 (or 80 in the case of unsecured connections), which is beneficial for environments that block non-web Internet connections using a firewall. Additionally, WebSocket enables streams of messages on top of TCP.

TCP alone deals with streams of bytes with no inherent concept of a message. Similar two-way browser–server communications have been achieved in non-standardized ways using stopgap technologies such as Comet or Adobe Flash Player.^[4]

Most browsers support the protocol, including Google Chrome, Firefox, Microsoft Edge, Internet Explorer, Safari and Opera.^[5]

The WebSocket protocol specification defines ws (WebSocket) and wss (WebSocket Secure) as two new uniform resource identifier (URI) schemes^[6] that are used for unencrypted and encrypted connections respectively. Apart from the scheme name and fragment (i.e. # is not supported), the rest of the URI components are defined to use URI generic syntax.^[7]

History

WebSocket was first referenced as TCPConnection in the HTML5 specification, as a placeholder for a TCP-based socket API.^[8] In June 2008, a series of discussions were led by Michael Carter that resulted in the first version of the protocol known as WebSocket.^[9] Before WebSocket, port 80 full-duplex communication was attainable using Comet channels; however, Comet implementation is nontrivial, and due to the TCP handshake and HTTP header overhead, it is inefficient for small messages. The WebSocket protocol aims to solve these problems without compromising the security assumptions of the web. The name "WebSocket" was coined by Ian Hickson and Michael Carter shortly thereafter through collaboration on the #whatwg IRC chat room,^[10] and subsequently authored for inclusion in the HTML5 specification by Ian Hickson. In December 2009, Google Chrome 4 was the first browser to ship full support for the standard, with WebSocket enabled by default.^[11] Development of the WebSocket protocol was subsequently moved from the W3C and WHATWG group to the IETF in February 2010, and authored for two revisions under Ian Hickson.^[12]

After the protocol was shipped and enabled by default in multiple browsers, the RFC 6455 (<https://www.rfc-editor.org/rfc/rfc6455>) was finalized under Ian Fette in December 2011.

WebSocket



The WebSocket logo

International standard	<u>RFC 6455</u> (https://www.rfc-editor.org/rfc/rfc6455) <u>WebSockets</u> (https://websockets.spec.whatwg.org)
Developed by	<u>IETF</u> <u>WHATWG</u>
Introduced	December 2011
Industry	<u>Computer science</u>
Connector type	<u>TCP</u>

RFC 7692 (<https://www.rfc-editor.org/rfc/rfc7692>) introduced compression extension to WebSocket using the DEFLATE algorithm on a per-message basis.

Web API

A web application (e.g. web browser) may use the WebSocket interface to maintain bidirectional communications with a WebSocket server.^[13]

Client example

In TypeScript.

```
// Connect to server
const ws = new WebSocket("wss://game.example.com/scoreboard");

// Receive ArrayBuffer instead of Blob
ws.binaryType = "arraybuffer";

// Set event listeners

ws.onopen = () => {
  console.log("Connection opened");
  ws.send("Hi server, please send me the score of yesterday's game");
};

ws.onmessage = (event: MessageEvent) => {
  console.log("Data received", event.data);
  ws.close(); // We got the score so we don't need the connection anymore
};

ws.onclose = (event: CloseEvent) => {
  console.log("Connection closed", event.code, event.reason, event.wasClean);
};

ws.onerror = () => {
  console.log("Connection closed due to error");
};
```

WebSocket interface

Type	Name ^[14]	Description
<u>Constructor</u>	<code>ws = new WebSocket(url [, protocols])</code>	Start opening handshake.^[15] ▪ url: A string containing: ▪ Scheme: must be ws, wss, http or https. ▪ Host. ▪ Optional port: If not specified, 80 is used for ws and http, and 443 for wss and https. ▪ Optional path. ▪ Optional query. ▪ No fragment. ▪ Optional protocols: A string or an array of strings used as the value of the Sec-WebSocket-Protocol header in the opening handshake. Exceptions: ▪ SyntaxError: ▪ url parsing^[a] failed. ▪ url has an invalid scheme. ▪ url has a fragment. ▪ protocols has duplicate strings.
	<code>ws.send(data)</code>	Send data message.^[16] ▪ data: must be string, Blob, ArrayBuffer or ArrayBufferView. Return: undefined. Exceptions: ▪ InvalidStateError: ws.readyState is CONNECTING. Note: If the data cannot be sent (e.g. because it would need to be buffered but the buffer is full), the connection is closed and the error event is fired.
	<code>ws.close([code] [, reason])</code>	Start closing handshake.^[17] ▪ Optional code: If specified, must be 1000 (<i>Normal closure</i>) or in the range 3000 to 4999 (application-defined). Defaults to 1000. ▪ Optional reason: If specified, must be a string whose UTF-8 encoding is up to 123 bytes. Defaults to an empty string. Return: undefined. Exceptions: ▪ InvalidAccessError: code is not 1000 nor is in the range 3000 to 4999. ▪ SyntaxError: UTF-8-encoded reason is longer than 123 bytes. Note:

		▪ If <code>ws.readyState</code> is OPEN or CONNECTING, <code>ws.readyState</code> is set to CLOSING and the closing handshake starts. ▪ If <code>ws.readyState</code> is CLOSING or CLOSED, nothing happens (because the closing handshake has already started).
Event	<pre>ws.onopen = (event) => {} ws.addEventListener("open", (event) => {})</pre>	Opening handshake succeeded. event type is Event.
	<pre>ws.onmessage = (event) => {} ws.addEventListener("message", (event) => {})</pre>	Data message received. event type is MessageEvent. This event is only fired if <code>ws.readyState</code> is OPEN. ^[18] ▪ <code>event.data</code> is the data received, of type: ▪ string for text. ▪ Blob or ArrayBuffer for binary (see <code>ws.binaryType</code>). ▪ <code>event.origin</code> is <code>ws.url</code> but only with the scheme, host, and port (if any).
	<pre>ws.onclose = (event) => {} ws.addEventListener("close", (event) => {})</pre>	The underlying TCP connection closed. event type is CloseEvent. ^{[19][20][21][22]} ▪ <code>event.code</code>: status code (integer). ▪ <code>event.reason</code>: reason for closing (string). ▪ <code>event.wasClean</code>: true if the TCP connection was closed after the closing handshake was completed, else false. Note: ▪ If the received <i>Close</i> frame contains a payload, <code>event.code</code> and <code>event.reason</code> get their value from the payload. ▪ If the received <i>Close</i> frame contains no payload, <code>event.code</code> is 1005 (<i>No code received</i>) and <code>event.reason</code> is an empty string. ▪ If no Close frame was received, <code>event.code</code> is 1006 (<i>Connection closed abnormally</i>) and <code>event.reason</code> is an empty string.
	<pre>ws.onerror = (event) => {} ws.addEventListener("error", (event) => {})</pre>	Connection closed due to error. event type is Event.
Attribute	<code>ws.binaryType</code> (string)	Type of event.data in ws.onmessage when a binary data message is received. Initially set to "blob" (Blob object). May be changed to "arraybuffer" (ArrayBuffer object). ^[23]
Read-only attribute	<code>ws.url</code> (string)	URL given to the WebSocket constructor with the following transformations: ▪ If scheme is http or https, change it to ws or wss respectively.
	<code>ws.bufferedAmount</code> (number)	Number of bytes of application data (UTF-8 text and binary data) that have been queued using <code>ws.send()</code> but not yet transmitted to the network . It resets to zero once all queued data has been sent. If the connection closes,

		this value will only increase, with each call to <code>ws.send()</code> , and never reset to zero. ^[24]
	<code>ws.protocol</code> (string)	Protocol accepted by the server , or an empty string if protocols was not specified in the WebSocket constructor.
	<code>ws.extensions</code> (string)	Extensions accepted by the server.
	<code>ws.readyState</code> (number)	Connection state. It is one of the constants below. Initially set to <code>CONNECTING</code> . ^[25]
Constant	<code>WebSocket.CONNECTING</code> = 0	Opening handshake is currently in progress. The initial state of the connection. ^{[26][27]}
	<code>WebSocket.OPEN</code> = 1	Opening handshake succeeded. The client and server may send messages to each other. ^{[28][29]}
	<code>WebSocket.CLOSING</code> = 2	Closing handshake is currently in progress. Either <code>ws.close()</code> was called or a <i>Close</i> message was received. ^{[30][31]}
	<code>WebSocket.CLOSED</code> = 3	The underlying TCP connection is closed. ^{[32][19][20]}

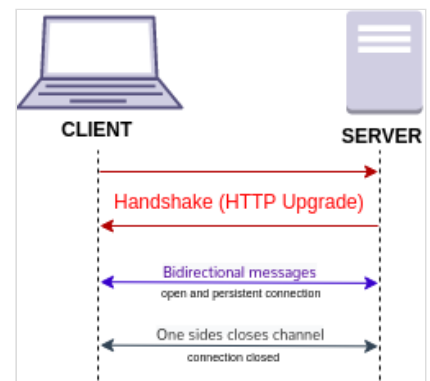
Protocol

Steps:

1. **Opening handshake:** HTTP request and HTTP response.
2. **Frame-based message exchange:** data, ping and pong messages.
3. **Closing handshake:** close message (request then echoed in response).

Opening handshake

The client sends an **HTTP request (method GET, version ≥ 1.1)** and the server returns an **HTTP response with status code 101** (*Switching Protocols*) on success. HTTP and WebSocket clients can connect to a server using the same port because the opening handshake uses HTTP. Sending additional HTTP headers (that are not in the table below) is allowed. HTTP headers may be sent in any order. After the *Switching Protocols* HTTP response, the opening handshake is complete, the HTTP protocol stops being used, and communication switches to a binary frame-based protocol.^{[33][34]}



A diagram describing a connection using WebSocket

HTTP headers relevant to the opening handshake

Side	Header	Value	Mandatory
Request	Origin ^[35]	Varies	Yes (for browser clients)
	Host ^[36]	Varies	Yes
	Sec-WebSocket-Version ^[37]	13	
	Sec-WebSocket-Key ^[38]	base64-encode (16 random bytes)	
Response	Sec-WebSocket-Accept ^[39]	base64-encode (SHA1(Sec-WebSocket-Key + " 258EAF5-E914-47DA-95CA-C5AB0DC85B11 ")) ^[b]	
Both	Connection ^{[40][41]}	<i>Upgrade</i>	
	Upgrade ^{[42][43]}	<i>websocket</i>	
	Sec-WebSocket-Protocol ^[44]	The request may contain a comma-separated list of strings (ordered by preference) indicating application-level protocols (built on top of WebSocket data messages) the client wishes to use. If the client sends this header, the server response must be one of the values from the list.	No
	Sec-WebSocket-Extensions ^{[45][46][47][48]}	Used to negotiate protocol-level extensions. The client may request extensions to the WebSocket protocol by including a comma-separated list of extensions (ordered by preference). Each extension may have a parameter (e.g. foo=4). The server may accept some or all extensions requested by the client. This field may appear multiple times in the request (logically equivalent to a single occurrence containing all values) and must not appear more than once in the response.	

Example request:^[34]

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

Example response:^[34]

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
Sec-WebSocket-Protocol: chat
```

The following [Python](#) code generates a random [Sec-WebSocket-Key](#).

```
import base64
import os

print(base64.b64encode(os.urandom(16)))
```

The following Python code calculates [Sec-WebSocket-Accept](#) using [Sec-WebSocket-Key](#) from the example request above.

```
import base64
import hashlib

KEY: bytes = b"dGhlIHNhbXBsZSBub25jZQ=="
MAGIC: bytes = b"258EAF5-E914-47DA-95CA-C5AB0DC85B11"
print(base64.b64encode(hashlib.sha1(KEY + MAGIC).digest()))
```

[Sec-WebSocket-Key](#) and [Sec-WebSocket-Accept](#) are intended to prevent a [caching proxy](#) from re-sending a previous WebSocket conversation,^[49] and does not provide any authentication, privacy, or integrity.

Though some servers accept a short `Sec-WebSocket-Key`, many modern servers will reject the request with error "invalid Sec-WebSocket-Key header".

Frame-based message

After the opening handshake, the client and server can, at any time, send **data messages** (text or binary) and **control messages** (*Close, Ping, Pong*) to each other. A message is composed of **one frame if unfragmented** or **at least two frames if fragmented**.

Fragmentation splits a message into **two or more frames**. It enables sending messages with initial data available but complete length unknown. Without fragmentation, the whole message must be sent in one frame, so the complete length is needed before the first byte can be sent, which requires a buffer.^[50] It was proposed to extend this feature to enable multiplexing several streams simultaneously (e.g. to avoid monopolizing a socket for a single large payload), but the protocol extension was never accepted.^[51]

- An **unfragmented message** consists of one frame with **FIN = 1** and **opcode ≠ 0**.
- A **fragmented message** consists of one frame with **FIN = 0** and **opcode ≠ 0**, followed by zero or more frames with **FIN = 0** and **opcode = 0**, and terminated by one frame with **FIN = 1** and **opcode = 0**.

Frame structure

Offset (bits)	Field ^[52]		Size (bits)	Description
0	FIN ^[53]		1	▪ 1 = final frame of a message. ▪ 0 = message is fragmented and this is not the final frame.
1	RSV1		1	Reserved. Must be 0 unless defined by an extension. If a non-zero value is received and none of the negotiated extensions defines the meaning of such a non-zero value, the connection must be closed. ^[54]
2	RSV2		1	
3	RSV3		1	
4	Opcode		4	See opcodes below.
8	Masked ^[55]		1	▪ 1 = frame is masked (i.e. masking key is present and the payload has been XORed with the masking key). ▪ 0 = frame is not masked (i.e. masking key is not present). See client-to-server masking below.
9	Payload length ^[56]		7, 7+16 or 7+64	Length of the payload (extension data + application data) in bytes. ▪ 0–125 = This is the payload length. ▪ 126 = The following 16 bits are the payload length. ▪ 127 = The following 64 bits (MSB must be 0) are the payload length. Endianness is big-endian . Signedness is unsigned . The minimum number of bits must be used to encode the length.
Varies	Masking key ^[57]		0 or 32	Random nonce . Present if the masked field is 1. The client generates a masking key for every masked frame.
	Payload	Extension data	Payload length (bytes)	Must be empty unless defined by an extension.
		Application data		Depends on the opcode

Opcodes

Frame type ^[58]		Opcode ^[59]	Related <u>Web API</u>	Description	Purpose	Fragmentable	Max. payload length (bytes)
Continuation frame		0		Non-first frame of a fragmented message.	Message fragmentation		2 ⁶³ − 1 ^[c]
Non- control frame	Text	1	send (), onmessage	UTF-8-encoded text.	Data message	Yes	
	Binary	2		Binary data.			
			3–7		Reserved for further non-control frames. May be defined by an extension. ^[60]		
Control frame ^[61]	Close	8	close (), onclose	The WebSocket closing handshake starts upon either sending or receiving a <i>Close</i> frame. ^[62] It may prevent data loss by complementing the TCP closing handshake. ^[63] No frame can be sent after sending a <i>Close</i> frame. If a <i>Close</i> frame is received and no prior <i>Close</i> frame was sent, a <i>Close</i> frame must be sent in response (typically echoing the status code received). The payload is optional, but if present, it must start with a two-byte big-endian unsigned integer status code , optionally followed by a UTF-8-encoded reason message not longer than 123 bytes. ^[64]	Protocol state	No	125
	Ping	9		May be used for <u>latency</u> measurement, <u>keepalive</u> and <u>heartbeat</u> . Both sides can send a ping (with any payload). Whoever receives it must, as soon as is practical, send back a pong with the same payload . A pong should be ignored if no prior ping was sent. ^{[65][66][67]}			
	Pong	10					
		11–15		Reserved for further control frames. May be defined by an extension. ^[60]			

Client-to-server masking

A **client must mask** all frames sent to the server. A **server must not mask** any frames sent to the client.^[68] Frame masking applies **XOR between the payload and the masking key**. The following pseudocode describes the algorithm used to both mask and unmask a frame.^[57]

```

for i from 0 to payload_length - 1
  payload[i] := payload[i] xor masking_key[i mod 4]

```


Status codes

Range ^[69]	Allowed in Close frame	Code ^[70]	Description
0–999	No		Unused
1000–2999 (Protocol)	Yes	1000	Normal closure.
		1001	Going away (e.g. browser tab closed; server going down).
		1002	Protocol error.
		1003	Unsupported data (e.g. endpoint only understands text but received binary).
	No	1004	Reserved for future usage
		1005	No code received.
		1006	Connection closed abnormally (i.e. closing handshake did not occur).
	Yes	1007	Invalid payload data (e.g. non UTF-8 data in a text message).
		1008	Policy violated.
		1009	Message too big.
		1010	Unsupported extension. The client should write the extensions it expected the server to support in the payload.
		1011	Internal server error.
	No	1015	TLS handshake failure.
3000–3999	Yes		Reserved for libraries, frameworks and applications. Registered directly with IANA .
4000–4999			Private use.

Compression extension

The `permessage-deflate` extension allows data messages to be compressed using the [DEFLATE](#) algorithm. For example, during the opening handshake, the client and server may use the following header to enable the extension. The `RSV1` field of the first frame of a data message must be set to indicate the payload data is compressed.^[71]

```
Sec-WebSocket-Extensions: permessage-deflate
```

Server implementation example

In Python.

Note: `recv()` returns up to the amount of bytes requested. For readability, the code ignores that, thus it may fail in non-ideal network conditions.

```
import base64
import hashlib
import struct
from typing import Optional
from socket import socket as Socket

def handle_websocket_connection(ws: Socket) -> None:
    # Accept connection
    conn, addr = ws.accept()

    # Receive and parse HTTP request
    key: Optional[bytes] = None
    for line in conn.recv(4096).split(b"\r\n"):
        if line.startswith(b"Sec-WebSocket-Key"):
            key = line.split()[-1]

    if key is None:
        raise ValueError("Sec-WebSocket-Key not found")

    # Send HTTP response
    sec_accept = base64.b64encode(hashlib.sha1(key + b"258EAF5E914-47DA-95CA-C5AB0DC85B11").digest())
    conn.sendall()
```

```

        b"\r\n".join([
            b"HTTP/1.1 101 Switching Protocols",
            b"Connection: Upgrade",
            b"Upgrade: websocket",
            b"Sec-WebSocket-Accept: " + sec_accept,
            b"",
            b""
        ])
    )

    # Decode and print frames
    while True:
        byte0, byte1 = conn.recv(2)
        fin: int = byte0 >> 7
        opcode: int = byte0 & 0b1111
        masked: int = byte1 >> 7
        assert masked, "The client must mask all frames"
        if opcode >= 8:
            assert fin, "Control frames are unfragmentable"

        # Payload size
        payload_size: int = byte1 & 0b111_1111
        if payload_size == 126:
            payload_size, = struct.unpack(">H", conn.recv(2))
            assert payload_size > 125, "The minimum number of bits must be used"
        elif payload_size == 127:
            payload_size, = struct.unpack(">Q", conn.recv(8))
            assert payload_size > 2**16-1, "The minimum number of bits must be used"
            assert payload_size <= 2**63-1, "The most significant bit must be zero"
        if opcode >= 8:
            assert payload_size <= 125, "Control frames must have up to 125 bytes"

        # Unmask
        masking_key: bytes = conn.recv(4)
        payload: bytearray = bytearray(conn.recv(payload_size))
        for i in range(payload_size):
            payload[i] = payload[i] ^ masking_key[i % 4]

        print("Received frame", FIN, opcode, payload)

if __name__ == "__main__":
    # Accept TCP connection on any interface at port 80
    ws: Socket = Socket()
    ws.bind(("", 80))
    ws.listen()

    handle_websocket_connection(ws)

```

Browser support

A secure version of the WebSocket protocol is implemented in Firefox 6,^[72] Safari 6, Google Chrome 14,^[73] Opera 12.10 and Internet Explorer 10.^[74] A detailed protocol test suite report^[75] lists the conformance of those browsers to specific protocol aspects.

An older, less secure version of the protocol was implemented in Opera 11 and Safari 5, as well as the mobile version of Safari in iOS 4.2.^[76] The BlackBerry Browser in OS7 implements WebSockets.^[77] Because of vulnerabilities, it was disabled in Firefox 4 and 5,^[78] and Opera 11.^[79] Using browser developer tools, developers can inspect the WebSocket handshake as well as the WebSocket frames.^[80]

Protocol version	Draft date	Internet Explorer	Firefox ^[81] (PC)	Firefox (Android)	Chrome (PC, Mobile)	Safari (Mac, iOS)	Opera (PC, Mobile)	Android Browser
hixie-75 (https://tools.ietf.org/html/draft-hixie-thewebsocketprotocol-75)	February 4, 2010				4	5.0.0		
hixie-76 (https://tools.ietf.org/html/draft-hixie-thewebsocketprotocol-76) hybi-00 (https://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol-00)	May 6, 2010 May 23, 2010		4.0 (disabled)		6	5.0.1	11.00 (disabled)	
hybi-07 (https://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol-07) , v7	April 22, 2011		6 ^[82] [d]					
hybi-10 (https://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol-10) , v8	July 11, 2011		7 ^[84] [d]	7	14 ^[85]			
RFC 6455 (https://www.rfc-editor.org/rfc/rfc6455), v13	December, 2011	10 ^[86]	11	11	16 ^[87]	6	12.10 ^[88]	4.4

Server implementations

- Nginx has supported WebSockets since 2013, implemented in version 1.3.13^[89] including acting as a reverse proxy and load balancer of WebSocket applications.^[90]
- Apache HTTP Server has supported WebSockets since July, 2013, implemented in version 2.4.5^[91]^[92]
- Internet Information Services added support for WebSockets in version 8 which was released with Windows Server 2012.^[93]
- lighttpd has supported WebSockets since 2017, implemented in lighttpd 1.4.46.^[94] lighttpd mod_proxy can act as a reverse proxy and load balancer of WebSocket applications. lighttpd mod_wstunnel can act as a WebSocket endpoint to transmit arbitrary data, including in JSON format, to a backend application. lighttpd supports WebSockets over HTTP/2 since 2022, implemented in lighttpd 1.4.65.^[95]
- Eclipse Mosquitto (<https://mosquitto.org/>) This is an MQTT broker, but it supports the MQTT over WebSocket. So, it can be considered a type of WebSocket implementation.

ASP.NET Core have support for WebSockets using the `app.UseWebSockets()` ; middleware.^[96]

Security considerations

Unlike regular cross-domain HTTP requests, WebSocket requests are not restricted by the same-origin policy. Therefore, WebSocket servers must validate the "Origin" header against the expected origins during connection establishment, to avoid cross-site WebSocket hijacking attacks (similar to cross-site request forgery), which might be possible when the connection is authenticated with cookies or HTTP authentication. It is better to use tokens or similar protection mechanisms to authenticate the WebSocket connection when sensitive (private) data is being transferred over the WebSocket.^[97] A live example of vulnerability was seen in 2020 in the form of Cable Haunt.

Proxy traversal

WebSocket protocol client implementations try to detect whether the user agent is configured to use a proxy when connecting to destination host and port, and if it is, uses HTTP CONNECT method to set up a persistent tunnel.

While the WebSocket protocol itself is unaware of proxy servers and firewalls, it features an HTTP-compatible handshake, thus allowing HTTP servers to share their default HTTP and HTTPS ports (80 and 443 respectively) with a WebSocket gateway or server. The WebSocket protocol defines a ws:// and wss:// prefix to indicate a WebSocket and a WebSocket Secure connection respectively. Both schemes use an HTTP upgrade mechanism to upgrade to the WebSocket protocol. Some proxy servers are

transparent and work fine with WebSocket; others will prevent WebSocket from working correctly, causing the connection to fail. In some cases, additional proxy-server configuration may be required, and certain proxy servers may need to be upgraded to support WebSocket.

If unencrypted WebSocket traffic flows through an explicit or a transparent proxy server without WebSockets support, the connection will likely fail.^[98]

If an encrypted WebSocket connection is used, then the use of Transport Layer Security (TLS) in the WebSocket Secure connection ensures that an HTTP CONNECT command is issued when the browser is configured to use an explicit proxy server. This sets up a tunnel, which provides low-level end-to-end TCP communication through the HTTP proxy, between the WebSocket Secure client and the WebSocket server. In the case of transparent proxy servers, the browser is unaware of the proxy server, so no HTTP CONNECT is sent. However, since the wire traffic is encrypted, intermediate transparent proxy servers may simply allow the encrypted traffic through, so there is a much better chance that the WebSocket connection will succeed if WebSocket Secure is used. Using encryption is not free of resource cost, but often provides the highest success rate, since it would be travelling through a secure tunnel.

A mid-2010 draft (version hixie-76) broke compatibility with reverse proxies and gateways by including eight bytes of key data after the headers, but not advertising that data in a `Content-Length: 8` header.^[99] This data was not forwarded by all intermediates, which could lead to protocol failure. More recent drafts (e.g., hybi-09^[100]) put the key data in a `Sec-WebSocket-Key` header, solving this problem.

See also

- | | |
|--|----------------------------------|
| ▪ <u>Comparison of WebSocket implementations</u> | ▪ <u>WebRTC</u> |
| ▪ <u>Network socket</u> | ▪ <u>HTTP/2</u> |
| ▪ <u>Push technology</u> | ▪ <u>Internet protocol suite</u> |
| ▪ <u>XMLHttpRequest</u> | ▪ <u>BOSH</u> |
| ▪ <u>Server-sent events</u> | |

Notes

- The **URL parsing algorithm** is described at <https://url.spec.whatwg.org/#concept-basic-url-parser>
- The plus sign represents **string concatenation**.
- The specification only explicitly restricts control frames. Thus, all other frames are restricted by the protocol limit of 63 bits for payload length (as the most significant bit must be zero).
- Gecko-based browsers versions 6–10 implement the WebSocket object as "MozWebSocket",^[83] requiring extra code to integrate with existing WebSocket-enabled code.

References

- "WebSockets Standard" (<https://websockets.spec.whatwg.org/>). *WHATWG WebSockets*. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/>) from the original on 2023-03-12. Retrieved 2022-05-16.
- "The WebSocket API" (<https://www.w3.org/TR/2021/NOTE-websockets-20210128/Overview.html>). *www.w3.org*. Archived (<https://web.archive.org/web/20220608221509/https://www.w3.org/TR/2021/NOTE-websockets-20210128/Overview.html>) from the original on 2022-06-08. Retrieved 2022-05-16.
- Ian Fette; Alexey Melnikov (December 2011). "Relationship to TCP and HTTP" (<https://datatracker.ietf.org/doc/html/rfc6455#section-1.7>). *RFC 6455 The WebSocket Protocol* (<https://www.rfc-editor.org/rfc/rfc6455>). IETF. sec. 1.7. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
- "Adobe Flash Platform – Sockets" (https://help.adobe.com/en_US/as3/dev/WSb2ba3b1aad8a27b0-181c51321220efd9d1c-8000.html). *help.adobe.com*. Archived (https://web.archive.org/web/20210418211002/https://help.adobe.com/en_US/as3/dev/WSb2ba3b1aad8a27b0-181c51321220efd9d1c-8000.html) from the original on 2021-04-18. Retrieved 2021-07-28. "TCP connections require a "client" and a "server". Flash Player can create client sockets."

5. "The WebSocket API (WebSockets)" (https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API). *MDN Web Docs*. Mozilla Developer Network. 2023-04-06. Archived (https://web.archive.org/web/20210728161324/http://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API) from the original on 2021-07-28. Retrieved 2021-07-26.
6. Graham Klyne, ed. (2011-11-14). "IANA Uniform Resource Identifier (URI) Schemes" (<https://www.iana.org/assignments/uri-schemes.html>). Internet Assigned Numbers Authority. Archived (<https://web.archive.org/web/20130425164755/http://www.iana.org/assignments/uri-schemes.html>) from the original on 2013-04-25. Retrieved 2011-12-10.
7. Ian Fette; Alexey Melnikov (December 2011). "WebSocket URIs" (<https://datatracker.ietf.org/doc/html/rfc6455#section-3>). *RFC 6455 The WebSocket Protocol* (<https://www.rfc-editor.org/rfc/rfc6455>). IETF. sec. 3. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
8. "HTML 5" (<https://www.w3.org/TR/2008/WD-html5-20080610/comms.html#tcp-connections>). *www.w3.org*. Archived (<https://web.archive.org/web/20160916071246/http://www.w3.org/TR/2008/WD-html5-20080610/comms.html#tcp-connections>) from the original on 2016-09-16. Retrieved 2016-04-17.
9. "[whatwg] TCPConnection feedback from Michael Carter on 2008-06-18 (whatwg.org from June 2008)" (<https://lists.w3.org/Archives/Public/public-whatwg-archive/2008Jun/0165.html>). *lists.w3.org*. Archived (<https://web.archive.org/web/20160427004936/https://lists.w3.org/Archives/Public/public-whatwg-archive/2008Jun/0165.html>) from the original on 2016-04-27. Retrieved 2016-04-17.
10. "IRC logs: freenode / #whatwg / 20080618" (<http://krijnhoetmer.nl/irc-logs/whatwg/20080618#l-1145>). *krijnhoetmer.nl*. Archived (<https://web.archive.org/web/20160821040755/http://krijnhoetmer.nl/irc-logs/whatwg/20080618#l-1145>) from the original on 2016-08-21. Retrieved 2016-04-18.
11. "Web Sockets Now Available In Google Chrome" (<https://blog.chromium.org/2009/12/web-sockets-now-available-in-google.html>). *Chromium Blog*. Archived (<https://web.archive.org/web/20211209195505/https://blog.chromium.org/2009/12/web-sockets-now-available-in-google.html>) from the original on 2021-12-09. Retrieved 2016-04-17.
12. <ian@hixie.ch>, Ian Hickson (6 May 2010). "The WebSocket protocol" (<https://tools.ietf.org/html/draft-hixie-thewebsocketprotocol-75>). *IETF Datatracker*. Archived (<https://web.archive.org/web/20170317201023/https://tools.ietf.org/html/draft-hixie-thewebsocketprotocol-75>) from the original on 2017-03-17. Retrieved 2016-04-17.
13. "Introduction" (<https://websockets.spec.whatwg.org/#network-intro>). *WHATWG WebSockets*. sec. 1.
14. "Interface definition" (<https://websockets.spec.whatwg.org/#interface-definition>). *WHATWG WebSockets*. sec. 3.1. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#interface-definition>) from the original on 2023-03-12. Retrieved 2024-04-10.
15. "new WebSocket(url, protocols)" (<https://websockets.spec.whatwg.org/#dom-websocket-websocket>). *WHATWG WebSockets*. sec. 3.1. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#dom-websocket-websocket>) from the original on 2023-03-12. Retrieved 2024-04-30.
16. "send(data)" (<https://websockets.spec.whatwg.org/#dom-websocket-send>). *WHATWG WebSockets*. sec. 3.1.
17. "close(code, reason)" (<https://websockets.spec.whatwg.org/#dom-websocket-close>). *WHATWG WebSockets*. sec. 3.1. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#dom-websocket-close>) from the original on 2023-03-12. Retrieved 2024-04-10.
18. "When a WebSocket message has been received" (<https://websockets.spec.whatwg.org/#:~:text=When%20a%20WebSocket%20message%20has%20been%20received>). *WHATWG WebSockets*. sec. 4.
19. "When the WebSocket connection is closed; substep 3" (<https://websockets.spec.whatwg.org/#closeWebSocket>). *WHATWG WebSockets*. sec. 4. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#closeWebSocket>) from the original on 2023-03-12. Retrieved 2024-04-13.
20. *The WebSocket Connection is Closed* (<https://www.rfc-editor.org/rfc/rfc6455#section-7.1.4>). IETF. sec. 7.1.4. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
21. *The WebSocket Connection Close Code* (<https://www.rfc-editor.org/rfc/rfc6455#section-7.1.5>). IETF. sec. 7.1.5. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
22. *The WebSocket Connection Close Reason* (<https://www.rfc-editor.org/rfc/rfc6455#section-7.1.6>). IETF. sec. 7.1.6. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
23. "socket.binaryType" (<https://websockets.spec.whatwg.org/#:~:text=socket.binaryType>). *WHATWG WebSockets*. sec. 3.1.
24. "socket.bufferedAmount" (<https://websockets.spec.whatwg.org/#:~:text=socket.bufferedAmount>). *WHATWG WebSockets*. sec. 3.1.
25. "ready state" (<https://websockets.spec.whatwg.org/#websocket-ready-state>). *WHATWG WebSockets*. sec. 3.1.
26. "CONNECTING" (<https://websockets.spec.whatwg.org/#ref-for-dom-websocket-connecting%E2%91%A0>). *WHATWG WebSockets*. sec. 3.1. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#ref-for-dom-websocket-connecting%E2%91%A0>) from the original on 2023-03-12. Retrieved 2024-04-13.

27. *Client Requirements* (<https://www.rfc-editor.org/rfc/rfc6455#section-4.1>). IETF. p. 14. sec. 4.1. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
28. "OPEN" (<https://websockets.spec.whatwg.org/#dom-websocket-open>). WHATWG WebSockets. sec. 3.1. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#dom-websocket-open>) from the original on 2023-03-12. Retrieved 2024-04-10.
29. *The WebSocket Connection is Established* (<https://www.rfc-editor.org/rfc/rfc6455#page-20>). IETF. p. 20. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
30. "CLOSING" (<https://websockets.spec.whatwg.org/#dom-websocket-closing>). WHATWG WebSockets. sec. 3.1. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#dom-websocket-closing>) from the original on 2023-03-12. Retrieved 2024-04-10.
31. *The WebSocket Closing Handshake is Started* (<https://www.rfc-editor.org/rfc/rfc6455#section-7.1.3>). IETF. sec. 7.1.3. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
32. "CLOSED" (<https://websockets.spec.whatwg.org/#dom-websocket-closed>). WHATWG WebSockets. sec. 3.1. Archived (<https://web.archive.org/web/20230312232711/https://websockets.spec.whatwg.org/#dom-websocket-closed>) from the original on 2023-03-12. Retrieved 2024-04-10.
33. *Opening Handshake* (<https://www.rfc-editor.org/rfc/rfc6455#section-1.3>). IETF. sec. 1.3. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
34. *Protocol Overview* (<https://www.rfc-editor.org/rfc/rfc6455#section-1.2>). IETF. sec. 1.2. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
35. *Client requirement 8*. (<https://www.rfc-editor.org/rfc/rfc6455#page-18>) IETF. p. 18. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
36. *Client requirement 4*. (<https://www.rfc-editor.org/rfc/rfc6455#page-17>) IETF. p. 17. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
37. *Client requirement 9*. (<https://www.rfc-editor.org/rfc/rfc6455#page-18>) IETF. p. 18. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
38. *Client requirement 7*. (<https://www.rfc-editor.org/rfc/rfc6455#page-18>) IETF. p. 18. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
39. *Server step 5.4*. (<https://www.rfc-editor.org/rfc/rfc6455#page-24>) IETF. p. 24. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
40. *Client requirement 6*. (<https://www.rfc-editor.org/rfc/rfc6455#page-18>) IETF. p. 18. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
41. *Server step 5.3* (<https://www.rfc-editor.org/rfc/rfc6455#page-24>). IETF. p. 24. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
42. *Client requirement 5*. (<https://www.rfc-editor.org/rfc/rfc6455#page-17>) IETF. p. 17. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
43. *Server step 5.2* (<https://www.rfc-editor.org/rfc/rfc6455#page-24>). IETF. p. 24. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
44. *Client requirement 10*. (<https://www.rfc-editor.org/rfc/rfc6455#page-18>) IETF. p. 18. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
45. *Client requirement 11*. (<https://www.rfc-editor.org/rfc/rfc6455#page-19>) IETF. p. 19. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
46. *Sec-WebSocket-Extensions* (<https://www.rfc-editor.org/rfc/rfc6455#section-11.3.2>). IETF. sec. 11.3.2. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
47. *Extensions* (<https://www.rfc-editor.org/rfc/rfc6455#section-9>). IETF. sec. 9. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
48. *Negotiating Extensions* (<https://www.rfc-editor.org/rfc/rfc6455#section-9.1>). IETF. sec. 9.1. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
49. "Main Goal of WebSocket protocol" (<https://trac.tools.ietf.org/wg/hybi/trac/wiki/FAQ>). IETF. Archived (<https://web.archive.org/web/20160422042232/https://trac.tools.ietf.org/wg/hybi/trac/wiki/FAQ>) from the original on 22 April 2016. Retrieved 25 July 2015. "The computation [...] is meant to prevent a caching intermediary from providing a WS-client with a cached WS-server reply without actual interaction with the WS-server."
50. *Fragmentation* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.4>). IETF. sec. 5.4. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
51. John A. Tamplin; Takeshi Yoshino (2013). *A Multiplexing Extension for WebSockets* (<https://datatracker.ietf.org/doc/html/draft-ietf-hybi-websocket-multiplexing>). IETF. I-D draft-ietf-hybi-websocket-multiplexing.
52. *Base Framing Protocol* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.2>). IETF. sec. 5.2. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).

53. *FIN* (<https://www.rfc-editor.org/rfc/rfc6455#page-28>). IETF. p. 28. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
54. *RSV1, RSV2, RSV3* (<https://www.rfc-editor.org/rfc/rfc6455#page-28>). IETF. p. 28. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
55. *Mask* (<https://www.rfc-editor.org/rfc/rfc6455#page-29>). IETF. p. 29. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
56. *Payload length* (<https://www.rfc-editor.org/rfc/rfc6455#page-29>). IETF. p. 29. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
57. *Client-to-Server Masking* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.3>). IETF. sec. 5.3. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
58. *frame-opcode* (<https://www.rfc-editor.org/rfc/rfc6455#page-31>). IETF. p. 31. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
59. *Opcode* (<https://www.rfc-editor.org/rfc/rfc6455#page-29>). IETF. p. 29. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
60. *Extensibility* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.8>). IETF. sec. 5.8. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
61. *Control Frames* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.5>). IETF. sec. 5.5. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
62. *The WebSocket Closing Handshake is Started* (<https://www.rfc-editor.org/rfc/rfc6455#section-7.1.3>). IETF. sec. 7.1.3. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
63. *Closing Handshake* (<https://www.rfc-editor.org/rfc/rfc6455#section-1.4>). IETF. sec. 1.4. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
64. *Close* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.5.1>). IETF. sec. 5.5.1. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
65. *Ping* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.5.2>). IETF. sec. 5.5.2. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
66. *Pong* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.5.3>). IETF. sec. 5.5.3. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
67. "Ping and Pong frames" (<https://websockets.spec.whatwg.org/#ping-and-pong-frames>). WHATWG WebSockets.
68. *Overview* (<https://www.rfc-editor.org/rfc/rfc6455#section-5.1>). IETF. sec. 5.1. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
69. *Reserved Status Code Ranges* (<https://www.rfc-editor.org/rfc/rfc6455#section-7.4.2>). IETF. sec. 7.4.2. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
70. *Defined Status Codes* (<https://www.rfc-editor.org/rfc/rfc6455#section-7.4.1>). IETF. sec. 7.4.1. doi:10.17487/RFC6455 (<https://doi.org/10.17487%2FRFC6455>). RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455>).
71. *Compression Extensions for WebSocket* (<https://www.rfc-editor.org/rfc/rfc7692>). IETF. doi:10.17487/RFC7692 (<https://doi.org/10.17487%2FRFC7692>). RFC 7692 (<https://datatracker.ietf.org/doc/html/rfc7692>).
72. Dirkjan Ochtman (May 27, 2011). "WebSocket enabled in Firefox 6" (<https://web.archive.org/web/20120526230019/https://developer.mozilla.org/en/WebSockets>). Mozilla.org. Archived from the original (<https://developer.mozilla.org/en/WebSockets>) on 2012-05-26. Retrieved 2011-06-30.
73. "Chromium Web Platform Status" (<https://www.chromium.org/developers/web-platform-status>). Archived (<https://web.archive.org/web/20170304203008/http://www.chromium.org/developers/web-platform-status>) from the original on 2017-03-04. Retrieved 2011-08-03.
74. "WebSockets (Windows)" ([https://msdn.microsoft.com/en-us/library/ie/hh673567\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/ie/hh673567(v=vs.85).aspx)). Microsoft. 2012-09-28. Archived ([https://web.archive.org/web/20150325091443/https://msdn.microsoft.com/en-us/library/ie/hh673567\(v=vs.85\).aspx](https://web.archive.org/web/20150325091443/https://msdn.microsoft.com/en-us/library/ie/hh673567(v=vs.85).aspx)) from the original on 2015-03-25. Retrieved 2012-11-07.
75. "WebSockets Protocol Test Report" (<https://web.archive.org/web/20160922040928/http://autobahn.ws/testsuite/reports/clients/index.html>). Tavendo.de. 2011-10-27. Archived from the original (<http://autobahn.ws/testsuite/reports/clients/index.html>) on 2016-09-22. Retrieved 2011-12-10.
76. Katie Marsal (November 23, 2010). "Apple adds accelerometer, WebSockets support to Safari in iOS 4.2" (https://www.appleinsider.com/articles/10/11/23/apple_adds_accelerometer_websockets_support_to_safari_in_ios_4_2.html). AppleInsider.com. Archived (https://web.archive.org/web/20110301114256/http://www.appleinsider.com/articles/10/11/23/apple_adds_accelerometer_websockets_support_to_safari_in_ios_4_2.html) from the original on 2011-03-01. Retrieved 2011-05-09.
77. "Web Sockets API" (https://web.archive.org/web/20110610191150/http://docs.blackberry.com/en/developers/deliverables/29271/Web_Sockets_support_1582781_11.jsp). BlackBerry. Archived from the original (http://docs.blackberry.com/en/developers/deliverables/29271/Web_Sockets_support_1582781_11.jsp) on June 10, 2011. Retrieved 8 July 2011.

78. Chris Heilmann (December 8, 2010). "WebSocket disabled in Firefox 4" (<https://hacks.mozilla.org/2010/12/websockets-disabled-in-firefox-4/>). *Hacks.Mozilla.org*. Archived (<https://web.archive.org/web/20170306114251/https://hacks.mozilla.org/2010/12/websockets-disabled-in-firefox-4/>) from the original on 2017-03-06. Retrieved 2011-05-09.
79. Aleksander Aas (December 10, 2010). "Regarding WebSocket" (<https://web.archive.org/web/20101215010748/http://my.opera.com/chooseopera/blog/2010/12/10/regarding-websocket>). *My Opera Blog*. Archived from the original (<http://my.opera.com/chooseopera/blog/2010/12/10/regarding-websocket>) on 2010-12-15. Retrieved 2011-05-09.
80. Wang, Vanessa; Salim, Frank; Moskovits, Peter (February 2013). "APPENDIX A: WebSocket Frame Inspection with Google Chrome Developer Tools" (https://web.archive.org/web/20151231184436/http://my.safaribooksonline.com/book/-/9781430247401/appendix-a-inspecting-websocket-traffic/sec1_xhtml). *The Definitive Guide to HTML5 WebSocket*. Apress. ISBN 978-1-4302-4740-1. Archived from the original (http://my.safaribooksonline.com/book/-/9781430247401/appendix-a-inspecting-websocket-traffic/sec1_xhtml) on 31 December 2015. Retrieved 7 April 2013.
81. "WebSockets (support in Firefox)" (<https://web.archive.org/web/20120526230019/https://developer.mozilla.org/en/WebSockets>). *developer.mozilla.org*. Mozilla Foundation. 2011-09-30. Archived from the original (<https://developer.mozilla.org/en/WebSockets>) on 2012-05-26. Retrieved 2011-12-10.
82. "Bug 640003 - WebSockets - upgrade to ietf-06" (https://bugzilla.mozilla.org/show_bug.cgi?id=640003). Mozilla Foundation. 2011-03-08. Archived (https://web.archive.org/web/20210401044535/https://bugzilla.mozilla.org/show_bug.cgi?id=640003) from the original on 2021-04-01. Retrieved 2011-12-10.
83. "WebSockets - MDN" (<https://web.archive.org/web/20120526230019/https://developer.mozilla.org/en/WebSockets>). *developer.mozilla.org*. Mozilla Foundation. 2011-09-30. Archived from the original (<https://developer.mozilla.org/en/WebSockets>) on 2012-05-26. Retrieved 2011-12-10.
84. "Bug 640003 - WebSockets - upgrade to ietf-07(comment 91)" (https://bugzilla.mozilla.org/show_bug.cgi?id=640003#c91). Mozilla Foundation. 2011-07-22. Archived (https://web.archive.org/web/20210401044535/https://bugzilla.mozilla.org/show_bug.cgi?id=640003#c91) from the original on 2021-04-01. Retrieved 2011-07-28.
85. "Chromium bug 64470" (<https://code.google.com/p/chromium/issues/detail?id=64470>). *code.google.com*. 2010-11-25. Archived (<https://web.archive.org/web/20151231184436/https://code.google.com/p/chromium/issues/detail?id=64470>) from the original on 2015-12-31. Retrieved 2011-12-10.
86. "WebSockets in Windows Consumer Preview" (<https://blogs.msdn.com/b/ie/archive/2012/03/19/websockets-in-windows-consumer-preview.aspx>). *IE Engineering Team*. Microsoft. 2012-03-19. Archived (<https://web.archive.org/web/20150906135037/http://blogs.msdn.com/b/ie/archive/2012/03/19/websockets-in-windows-consumer-preview.aspx>) from the original on 2015-09-06. Retrieved 2012-07-23.
87. "WebKit Changeset 97247: WebSocket: Update WebSocket protocol to hybi-17" (<https://trac.webkit.org/changeset/97249>). *trac.webkit.org*. Archived (<https://web.archive.org/web/20120105121750/http://trac.webkit.org/changeset/97249>) from the original on 2012-01-05. Retrieved 2011-12-10.
88. "A hot Opera 12.50 summer-time snapshot" (<https://web.archive.org/web/20120805234006/http://my.opera.com/ODIN/blog/2012/08/03/a-hot-opera-12-50-summer-time-snapshot>). Opera Developer News. 2012-08-03. Archived from the original (<http://my.opera.com/ODIN/blog/2012/08/03/a-hot-opera-12-50-summer-time-snapshot>) on 2012-08-05. Retrieved 2012-08-03.
89. "Welcome to nginx!" (<https://web.archive.org/web/20120717224952/http://nginx.org/en/CHANGES>). *nginx.org*. Archived from the original (<http://nginx.org/en/CHANGES>) on 17 July 2012. Retrieved 3 February 2022.
90. "Using NGINX as a WebSocket Proxy" (<https://www.nginx.com/blog/websocket-nginx/>). *NGINX*. May 17, 2014. Archived (<https://web.archive.org/web/20191006062348/https://www.nginx.com/blog/websocket-nginx/>) from the original on October 6, 2019. Retrieved November 3, 2019.
91. "Overview of new features in Apache HTTP Server 2.4" (https://httpd.apache.org/docs/trunk/new_features_2_4.html). *Apache*. Archived (https://web.archive.org/web/20201111201741/http://httpd.apache.org/docs/trunk/new_features_2_4.html) from the original on 2020-11-11. Retrieved 2021-01-26.
92. "Changelog Apache 2.4" (<https://www.apachelounge.com/Changelog-2.4.html>). *Apache Lounge*. Archived (<https://web.archive.org/web/20210122162741/https://www.apachelounge.com/Changelog-2.4.html>) from the original on 2021-01-22. Retrieved 2021-01-26.
93. "IIS 8.0 WebSocket Protocol Support" (<https://docs.microsoft.com/en-us/iis/get-started/whats-new-in-iis-8/iis-80-websocket-protocol-support>). *Microsoft Docs*. 28 November 2012. Archived (<https://web.archive.org/web/20200218153735/https://docs.microsoft.com/en-us/iis/get-started/whats-new-in-iis-8/iis-80-websocket-protocol-support>) from the original on 2020-02-18. Retrieved 2020-02-18.
94. "Release-1.4.46 - Lighttpd - lighty labs" (https://redmine.lighttpd.net/projects/lighttpd/wiki/Release-1_4_46). Archived (https://web.archive.org/web/20210116155634/https://redmine.lighttpd.net/projects/lighttpd/wiki/Release-1_4_46) from the original on 2021-01-16. Retrieved 2020-12-29.
95. "Release-1.4.65 - Lighttpd - lighty labs" (https://redmine.lighttpd.net/projects/lighttpd/wiki/Release-1_4_65). Archived (https://web.archive.org/web/20240503061036/https://redmine.lighttpd.net/projects/lighttpd/wiki/Release-1_4_65) from the original on 2024-05-03. Retrieved 2024-05-03.
96. "WebSockets support in ASP.NET Core" (<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/websockets?view=aspnetcore-9.0>). *learn.microsoft.com*. Retrieved 2 May 2025.

97. Christian Schneider (August 31, 2013). "Cross-Site WebSocket Hijacking (CSWSH)" (<https://www.christian-schneider.net/CrossSiteWebSocketHijacking.html#main>). *Web Application Security Blog*. Archived (<https://web.archive.org/web/20161231061757/http://www.christian-schneider.net/CrossSiteWebSocketHijacking.html#main>) from the original on December 31, 2016. Retrieved December 30, 2015.
98. Peter Lubbers (March 16, 2010). "How HTML5 Web Sockets Interact With Proxy Servers" (<http://www.infoq.com/articles/Web-Sockets-Proxy-Servers>). *Infoq.com*. C4Media Inc. Archived (<https://web.archive.org/web/20160508202722/http://www.infoq.com/articles/Web-Sockets-Proxy-Servers>) from the original on 2016-05-08. Retrieved 2011-12-10.
99. Willy Tarreau (2010-07-06). "WebSocket -76 is incompatible with HTTP reverse proxies" (<https://www.ietf.org/mail-archive/web/hybi/current/msg02149.html>). *ietf.org* (email). Internet Engineering Task Force. Archived (<https://web.archive.org/web/20160917063052/http://www.ietf.org/mail-archive/web/hybi/current/msg02149.html>) from the original on 2016-09-17. Retrieved 2011-12-10.
100. Ian Fette (June 13, 2011). "Sec-WebSocket-Key" (<https://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol-09#section-11.4#section-11.4>). *The WebSocket protocol, draft hybi-09* (<https://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol-09#section-11.4>). IETF. sec. 11.4. Retrieved June 15, 2011. Archived (<https://web.archive.org/web/20160201000218/http://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol-09#section-11.4>) February 1, 2016, at the [Wayback Machine](#)

External links

- IETF Hypertext-Bidirectional (HyBi) working group (<https://datatracker.ietf.org/wg/hybi/charter/>)
 - RFC 6455 (<https://www.rfc-editor.org/rfc/rfc6455>) The WebSocket protocol – Proposed Standard published by the IETF HyBi Working Group
 - The WebSocket protocol (<https://tools.ietf.org/html/draft-ietf-hybi-thewebsocketprotocol>) – Internet-Draft published by the IETF HyBi Working Group
 - The WebSocket protocol (<https://tools.ietf.org/html/draft-hixie-thewebsocketprotocol-76>) – Original protocol proposal by Ian Hickson
 - The WebSocket API (<https://dev.w3.org/html5/websockets/>) Archived (<https://web.archive.org/web/20150607035919/http://dev.w3.org/html5/websockets/>) 2015-06-07 at the [Wayback Machine](#) – W3C [Working Draft](#) specification of the API
 - The WebSocket API (<http://www.w3.org/TR/websockets/>) – W3C [Candidate Recommendation](#) specification of the API
 - [WebSocket.org](https://www.websocket.org/) (<https://www.websocket.org/>) Archived (<https://web.archive.org/web/20180916101105/http://websocket.org/>) 2018-09-16 at the [Wayback Machine](#) WebSocket demos, loopback tests, general information and community
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=WebSocket&oldid=1365401917>"